

B.Sc. Thesis

Investigating ML-Based Solutions for SSD-Caching

Tareq Ahrari

Matriculation: 445448

Date: 23.09.2025

First Examiner: Prof. Redha Gouicem, Ph.D.

Second Examiner: Prof. Dr.-Ing. Klaus Wehrle

Advisor: Mostafa Hadizadeh

*This work was conducted at the Operating Systems Teaching and Research Unit
at the RWTH Aachen University.*

Affidavit

I, Tareq Ahrari, hereby certify that I have written this B.Sc. thesis titled **Investigating ML-Based Solutions for SSD-Caching** independently and without unauthorized outside help.

I have indicated all passages in the thesis that are taken from printed works or sources from the Internet, either in wording or in meaning, by appropriately citing the sources.

I also assure that all submitted versions, printed or electronic, are identical.

Place, Date

Aachen, 23.09.2025

t.ahrari Signature

Place, Date

Signature

Abstract

Flash caches in front of HDDs can reduce latency and backend load, but due to their structure, naive implementations like always-admit can significantly shorten device lifetime. We aim to investigate the use of hybrid caches in Meta’s Cachelib, using DRAM as a first layer and SSD as a second layer. We implement and compare traditional eviction policies such as LRU and TinyLFU paired with always-admit and reject-first as admission policies, against two machine learning based solutions, Baleen (admission) and MAT (eviction). We implement a hybrid cache using a 4 KiB block size, write-back, and `0_Direct`. Baleen is integrated as an admission policy on top, making decisions using a GBDT. MAT also queues a GBDT model, but on eviction, and uses a heuristic method as a filter. We evaluate these policies using MSR Cambridge block I/O traces, focusing on response time, hit rate, SSD writes, and peak disk-head time. We observe TinyLFU across workloads performing better than LRU. Furthermore, always-admit performs better than reject-first, but at the cost of more writes to the SSD. Baleen achieves a similar response time and hit rate to LRU with always-admit, and reduces SSD writes by 6%. Depending on the workload, Baleen either improves or worsens peak disk-head time, exhibiting strong workload and cache size sensitivity. MAT across workloads increases hit rate by 0,05%, and sometimes outperforms other algorithms. However, while the increase in hit rate is consistent, we observe a lot of volatility for other metrics. So while MAT increases its target metric, an overall performance increase is not guaranteed. Overall, we find that heuristic methods’ results are robust, and while machine learning based solutions can improve performance, results remain workload-dependent and do not generalize well.

Contents

1	Introduction	4
2	Background	7
2.1	Caching	7
2.1.1	Storage cache	8
2.1.2	Web caches	8
2.1.3	Prefetching	8
2.2	Storage architecture in web services	9
2.3	Backend bottleneck	9
2.4	Flash caches	9
2.5	Storage hierachies in modern systems	10
2.6	Challenges in Flash caching	10
2.6.1	Admission policy	10
2.6.2	Write Amplification	11
2.7	Cachelib	11
2.7.1	Design	11
2.7.2	Admission Policies	12
2.7.3	Eviction Policies	12
2.7.4	Navy	14
2.8	Machine learning based caching	15
2.8.1	Reinforcement learning	15
2.8.2	Supervised learning	15
2.8.3	Hybrid approach	16
2.8.4	Challenges of ML in caching	16
2.9	Related work	17
2.9.1	Baleen	17
2.9.2	MAT	18

2.9.3	GL-Cache	18
2.9.4	Raven	19
3	Design and Implementation	20
3.1	Architecture overview	20
3.1.1	Cache Architecture	20
3.1.2	Trace Replay Architecture	21
3.1.3	Cachelib	22
3.2	Baleen	23
3.2.1	Cachelib integration	25
3.3	MAT	25
3.3.1	Features	25
3.3.2	Read	25
3.3.3	Write	26
3.3.4	ML Model	26
4	Results	27
4.1	Methodology	27
4.1.1	Cloudlab	27
4.1.2	Workloads	27
4.1.3	Caching Policies	28
4.1.4	Baleen training	29
4.1.5	MAT training	30
4.1.6	Evaluation metrics	31
4.2	Results	32
4.3	Discussion	36
4.3.1	Heuristic methods	36
4.3.2	Baleen	36
4.3.3	MAT	37
4.4	Future work	38
4.5	Limitations	38
5	Conclusion	40
5.1	Research questions	40
5.2	Conclusion	41

Chapter 1

Introduction

SSD (Solid State Drive) caching has become an integral part of caching design, as it enhances I/O performance compared to HDDs (Hard Disk Drives), while also offering more storage capacity than DRAM (Dynamic Random Access Memory) at a similar cost. Modern data centers use bulk storage systems, backed by many HDDs, because of their low cost for a high amount of memory. SSD caches, also known as flash caches, are often used as a cache to absorb a fraction of the workload (ideally, the fraction that is most frequently accessed), compensating for HDDs' low performance. While flash caching reduces latency and achieves high throughput compared to HDD, it has limited write endurance, which constrains the amount of data to insert. Admitting everything increases hit ratio and latency, but also shortens device lifetime. Therefore, flash caching becomes a trade-off problem, weighing latency, hit ratio, and throughput against SSD lifetime.

Cachelib is a general-purpose caching engine designed to act as a cache for various layers and use cases, combining common functionalities and solving common problems. Navy is Cachelib's NVM caching engine, consisting of a first-level DRAM cache and a secondary flash cache, which features a block cache for large items and a hash cache for small items. Cachelib inserts items into DRAM, and when it evicts an item from DRAM, it either admits the item to the flash cache or writes it to the HDD. This two-level caching layer has proven itself successful[1]. However, Navy only employs heuristic methods, such as LRU and TinyLFU, or more complex ones, such as a dynamic reject.

Machine learning offers an attractive solution to the flash caching problem. Machine learning trains algorithms to learn patterns for achieving optimized results. We reformulate the flash caching problem as an optimization problem to maintain or improve performance while

minimizing SSD writes and preserving flash lifetime. ML solutions can help by learning items and item features, and thus predicting future access with a higher precision than heuristic methods. Through this approach, the cache can make smart decisions regarding which items to evict and admit. However, applying machine learning to caching is difficult. Furthermore, machine learning adds overhead that needs consideration when measuring performance. If a solution improves performance but imposes significant overhead, its usability compared to heuristic methods remains fragile.

In this thesis, we compare heuristic methods, such as LRU and TinyLFU, to machine learning solutions, including Baleen and MAT. We run all the algorithms on top of a hybrid navy cache. We use a two-level cache, with a DRAM cache on top, and a flash cache in front of the origin. We cache items at 4KiB block granularity. We evaluate admission and eviction policies; we do not evaluate any prefetching in this work. We further aim to understand whether a developed machine learning solution on top of cachelib can show clear improvements over heuristic methods. To do that, we answer the following questions:

1. Is there a meaningful difference between policies in Cachelib?
2. Can machine learning based policies outperform heuristic policies in Cachelib?
3. Are these differences coherent across different workloads?

To answer these questions, we use a setup consisting of:

1. **Cachelib.** We set up a flash-based NVMe cache using Cachelib. We use a C++ wrapper around Cachelib that replays traces with `O_DIRECT`, uses write-back on evictions, and a 4 KiB block size. For Baleen, we use a 2 MiB segment size.
2. **Policies.** The default admission policy is always-admit. We also evaluate LRU and TinyLFU as eviction policies combined with always-admit. In addition, we evaluate LRU and TinyLFU with the reject-first admission policy and with Baleen as the admission policy. Lastly, we test MAT as an eviction policy with always-admit.
3. **Evaluation metrics.** We measure response time, hit rate, and SSD writes. For Baleen, we also test peak disk-head time.

For Baleen, we use the official Baleen simulator (<https://github.com/wonglkd/Baleen-FAST24>). We train Baleen using the official training pipeline on 10% of the respective trace. We overwrite the admission policy in Cachelib, and upon admission, consult the trained model. Since

the model predicts on feature vectors, we create those at runtime. We implement a 2-pass for MAT, where the first pass involves generating training data and learning for the model. The second pass is the run, where for each eviction we consult the model, which computes each item's time-to-next-access (TTA) using feature vectors, and then returns a set of items ready for eviction. The idea is to reduce machine learning overhead by using heuristic methods (in our case LRU) as a filter and only using the model to make decisions after the filter. We used our standard trace implementation and extended it to use the necessary functions. MAT provides a fork of Cachelib, which we use for our implementation.

For this setup, we achieve the following results:

1. **TinyLFU vs LRU.** TinyLFU achieves a faster response time compared to LRU, while maintaining the same hit rate and writing slightly less to the SSD. We therefore observe a clear improvement when using TinyLFU.
2. **Always-admit vs Reject-first.** Using reject-first leads to a sharp decline in SSD writes (37,5%), but lowers hit rate and response time compared to always-admit. The use of an admission policy highlights the trade-off of performance vs. durability. While always-admit achieves better results, using reject-first means a significant reduction of writes to SSD.
3. **Baleen.** Baleen achieves similar response time, hit rate, and writes slightly less to SSD than LRU with always-admit. However, when testing for peak disk-head time (DT), we obtain mixed results for Baleen, with one instance significantly lowering peak DT, one instance increasing peak DT, and one instance acting similarly to the baseline. While Baleen can improve results, we do not observe a clear improvement across workloads, and sometimes even observe a worse performance. Moreover, we did not test Baleen on all workloads, as Baleen for large workloads (50 GB) could not be tested or resulted in model failure.
4. **MAT.** MAT improves the hit ratio across workloads by around 0,05%, but its response time can vary depending on the workload. For SSD writes, we observe similarity to the baseline. MAT improves hit rate, but overall performance remains fragile and is workload dependent.

Chapter 2

Background

2.1 Caching

Data is commonly stored on large-capacity devices, such as HDDs, due to their high storage capacity and relatively low cost, allowing them to store large amounts of data. However, due to their mechanical nature, accessing data from HDDs is slow, which is a problem for frequently accessed data, leading to performance issues. To mitigate this problem, research considers caching as a cost-efficient solution for designing high-performance data storage systems. Caching is a technique used to reduce latency and improve system performance by temporarily storing frequently accessed data in a faster, intermediate storage layer. The main idea is that data that is frequently accessed should be quickly available. For this purpose, data storage systems employ multi-level cache architectures that combine SSDs, DRAM, and NVM. Therefore, frequently used data can be stored in one of these layers, leading to faster access. However, DRAM and flash cache modules are limited in capacity, which prevents the storage of all data. Thus, caches must make decisions on what data to store (admission policy) and, if the cache is already full and new data is to be stored, what data to evict (eviction policy). These policies can significantly impact caching performance. Moreover, caches can proactively load data into the cache before they are requested. Caching relies on the principle of locality, consisting of temporal and spatial locality. Temporal locality data that has been recently accessed and is likely to be accessed again, and spatial locality refers to the principle that memory near recently accessed memory is more likely to be accessed again in the future [19].

2.1.1 Storage cache

For storage systems consisting of several layers, faster layers can be used as caches for slower layers. Modern data centers use SSDs as a caching layer for HDDs. More frequently or recently accessed blocks of disk memory can be cached to SSD, speeding up subsequent access. This hybrid caching solution makes use of the high speed of SSDs and the high capacity of HDDs [8].

2.1.2 Web caches

To improve the performance of web content delivery, web caches store resources close to the user. There are multiple ways to perform this, including at the user's browser or using a proxy cache, which is located between the user and the server to which the request is issued, and CDN caches, which are stationed on the provider's side. When a user requests some web content, instead of the request traveling all the way to the original web server, caches along the way can provide the content if available. This reduces bandwidth usage and latency, as users get the content faster without the need to inquire the original server, thus also reducing the load on the original server [15].

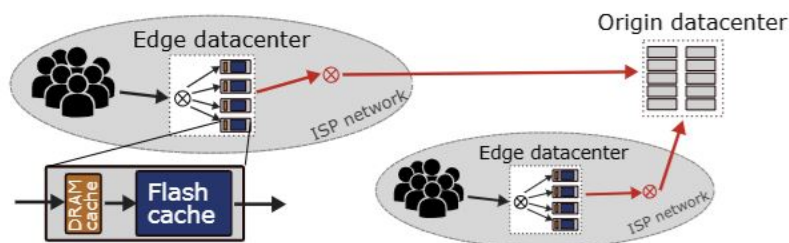


Figure 2.1: Example usage of CDN caches [20]

2.1.3 Prefetching

Prefetching refers to the concept of fetching data before a request for that data is issued, expecting a future access. Prefetching is the prediction of an object's future access and caching it if it is likely to be accessed again in the future. If we frequently access a block in cache, we can assume that blocks nearby it in memory will be accessed as well in the future. Thus, cache management systems decide to preemptively load data into the cache, even though it has not been accessed yet. Prefetching can reduce peak disk head time by lowering the workload on the HDD during its peak usage. It can also improve throughput by sending object earlier to cache, reducing future access to the slow HDD. However, prefetching

can also slow down our system by inserting objects with low value, which increases flash writes, response time, and uses cache resources [18, 23].

2.2 Storage architecture in web services

Web services frequently use caches. Modern services, such as Facebook’s Tectonic [14] use caches at almost every layer of their infrastructure, including CDN caches, social graph caches, application look-aside caches, and block storage system caches. In Tectonic, these caches were built and maintained separately, differing in implementation. This approach seems logical, because each cache needed to handle different workloads, so in order to work best on their respective workloads, using varying implementations makes the most sense. Berg et al. introduced a different view on Tectonic caching systems. Instead of having many caches, Berg et al. proposed Cachelib [1], a general-purpose caching engine, implemented based on common functionality between different caches. Cachelib provides a single caching system, making deployment and maintenance easier for caches. Moreover, it also provides hybrid caching, using DRAM and SSD. Modern web services employ caches to serve the bulk of requests from fast-level caches, significantly reducing backend time.

2.3 Backend bottleneck

HDDs are often the choice of backend storage because their cost is significantly lower than that of SSDs. However, HDDs are slower than SSDs, so serving the abundance of requests from HDDs results in higher latency and response time. Furthermore, hard disks have high access times and limited IOPS. By caching data in faster levels, such as DRAM or flash, services can avoid wearing out the disk I/O capacity. If the disk has insufficient I/O capacity, requests will accumulate and slow the system down. One such example can be seen in Tectonic, employing a flash cache in front of the HDDs, which stores popular items and results in reduced disk utilization.

2.4 Flash caches

HDDs are cheap but slow, so in order to improve latency and response time, caches often use SSDs. However, SSDs are more expensive for less storage. Furthermore, SSDs have limited write endurance, as each flash cell in the SSD can only be allocated a finite number of write/erase cycles. Using flash memory recklessly can thus shorten the lifetime of the device. In write-heavy use cases, this has even led to the limitation of using flash storage, as there is a

significant reduction in lifetime. Admission policies can reduce SSD wear by strictly dictating which items are allowed to cache, reducing the load on flash. For example, if the workload consists of many one-time accesses, a naive cache would admit all these accesses to the cache, but a more restrictive admission policy, such as reject-first, can reduce SSD writes and increase performance. flash caches often use a reject-first approach to increase flash lifetime. Recently, many experiments have used machine learning admission policies to learn the workload and to optimize performance further.

2.5 Storage hierarchies in modern systems

Modern datacenters employ caching to balance performance, cost, and capacity. For that, they employ hybrid caches. DRAM sits at the top, followed by SSD, and HDD is at the bottom. As HDDs are slow in nature, they can become a bottleneck for I/O intensive requests. As a consequence, modern datacenters using bulk storage systems rely on the top layer caches (DRAM and SSD) to absorb the bulk of the requested issues and reduce the load on the HDD. As such, optimizing caching at every layer is crucial. One of the key goals is to minimize latency and disk traffic by serving as many requests as possible to top-level caches. For example, at Netflix, 95% of its traffic is served by caches, significantly reducing the work of the original server. Another goal is the optimization of disk-head time (DT), meaning the fraction of the time the disk is busy. The reduction of peak disk-head time means a faster system that might need fewer disks to achieve throughput requirements. Therefore, reducing peak disk-head time has become an integral evaluation metric [14].

2.6 Challenges in Flash caching

SSD Caching is the layer between the DRAM and HDD, acting as a second-layer cache. Such caches are also often referred to as flash caches. Flash caches in large-scale storage systems, such as datacenters, have proven to be effective in reducing the number of HDDs required by shifting a fraction of the workload to SSD instead of HDD. Moreover, flash caches have been successful in acting as web caches by saving network bandwidth.

However, SSD caching comes with its own set of challenges and limitations.

2.6.1 Admission policy

Admission policies control which data is inserted into the cache. The goal is to reduce unnecessary or inefficient writes to SSD, while maintaining a similar hit rate and response time.

An ideal admission policy admits data that is accessed frequently and rejects data that is unlikely to be requested again. However, predicting future access can be difficult, especially for complex and large workloads. This can also lead to worse results, as a wrong admission policy can worsen the cache’s performance. Furthermore, using an admission policy incurs an additional overhead that needs to be accounted for.

2.6.2 Write Amplification

In flash, data can not be overwritten. Thus, when data is updated, the flash needs to provide a fresh page for the update, while erasing the old data, usually by using a garbage collection system. Moreover, when we write to flash, the physical amount of data is more than the logical amount of data previously intended to write. Cache management can help reduce write amplification by reducing the amount of data written to flash or by caching data that improves throughput. When the cache’s admission policy admits too much, the data floods the cache. This wears out the flash memory as well as downgrades performance. To avoid this, admission policies often employ a write rate limit, which limits the amount of writes issued to flash. However, this constraint can harm performance, as caching worthy data is overlooked. Therefore, the challenge becomes an optimization problem, to maximize hit rate under a write budget [12].

2.7 Cachelib

Cachelib is a general-purpose caching engine. The motivation for Cachelib stems from Facebook having several caches for different applications, which at one point became unmanageable, so Berg et al. implemented Cachelib based on common functionalities of the different caches. Furthermore, further functionality implementation on top of the cachelib engine is possible to facilitate different purposes. Cachelib natively supports multi-level caching by integrating both DRAM and flash layers. This architecture allows efficient management of hot and cold data, leveraging DRAM for fast access and flash for extended capacity [1].

2.7.1 Design

Cachelib provides a cachelib API that programmers can use to build additional layers on top of the cachelib engine. Data to be cached are items (either in DRAM or flash), and each item has a unique item handle through `w`. When created, a reference count is incremented and decremented when it is destroyed, preventing illegal deletion. Data can be allocated, replaced, deleted, updated, and accessed. Cachelib uses both DRAM and flash as storage. If an item is

to be cached, it will be cached in the DRAM. If the DRAM cache is full, DRAM will evict an item from its cache, which will either be admitted to flash or discarded. The flash consists of an LOC (Large Object Cache) for large items ($\geq 2\text{KB}$) and an SOC. (Small Object Cache) for smaller items ($< 2\text{KB}$) [1].

2.7.2 Admission Policies

Admission policies decide the storage of data blocks. Several policies exist, each different in its approach and results. Cachelib allows users to choose from a range of policies; we quickly go over some of the most prominent ones [1].

always-admit This admission policy always-admits data evicted from the first-level cache into the second-level cache. This is our baseline policy against which we will test other policies.

reject-first (LARC) reject-first or Lazy Adaptive Replacement Cache (LARC) is a policy that improves performance when the cache consists of lots of one-time accesses, meaning data that is accessed once but never again, therefore reducing SSD lifetime without future cache hits. Reject-first maintains a running window of accessed addresses and, during the window's period, does not admit items on their first access. However, accessing an item a second time during the running window results in an admission to flash [10].

Random Policy This policy can serve as both an admission and an eviction policy. It uses a probability factor (p) and admits each evicted block from the 1st-level to the 2nd-level with probability p [16].

Dynamic Reject Dynamic reject is a smart policy that adjusts its behavior based on the write traffic. It sets a daily write rate and monitors write traffic, and if traffic exceeds a specified threshold, it probabilistically rejects inserts [26].

2.7.3 Eviction Policies

Several eviction policies exist for both DRAM and flash. The DRAM cache consists of disjoint pools, each with its own eviction policy. Memory is allocated using 4MB size slabs holding objects of similar size and maintaining their own eviction policies states. Currently, supported eviction policies are LRU, 2Q, TinyLFU, and LRU. Furthermore, memory is rebalanced between slab classes to approximate a global eviction policy [1].

When the cache is full and new items arrive, eviction policies determine which item to evict. We briefly go over the most common ones.

LRU (Least recently used) LRU evicts the least recently used item to make room for a new one, since recently accessed items are more likely to be reused. Therefore, evicting the least-used items removes cold entries. LRU uses a list and keeps track of the recency order of access [4].

LFU (Least frequently used) LFU mirrors LRU but evicts the least frequently used item instead of the least recently used one, so frequency decides eviction [7].

TinyLFU TinyLFU is an LFU variant that tracks recent access frequency with a frequency sketch. When admitting an item, TinyLFU compares the candidate's frequency with the eviction candidates one and admits the item only if its frequency is higher. This way, one-hit and low-frequency items are prevented from being cached, allowing the caching of more frequently accessed items [6].

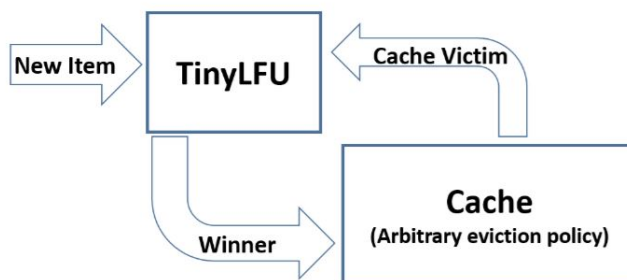


Figure 2.2: TinyLFU cache [6]

FIFO First In, First Out (FIFO) evicts the first item admitted to the cache. FIFO implements a queue, where every item moves when a new item is inserted into the queue. When the queue is full, the first item is removed, allowing the new item to be inserted at the beginning [16].

2Q The 2Q policy employs two structures, a queue called "A1" and a second queue called "Am". The cache first admits a new item to the A1 queue, which uses FIFO to admit and evict items. If the item is referenced again, it promotes the item to the Am queue, which uses LRU for eviction. This way, hot items reside longer in cache, while cold items are quickly evicted [11].

2.7.4 Navy

Navy is cachelibs NVM caching engine, using a hybrid cache. The first level is a small DRAM cache, and the second level is a bigger SSD cache. Before navy inserts items, it allocates space in DRAM. When the DRAM cache is full, according to its eviction policy, item(s) are removed. Then, the flash cache's admission policy decides on admission of the item. Items are either admitted to the LOC flash cache (if the size is $>2\text{kB}$) or else to SOC. If the flash cache is full, according to its eviction policy, items are evicted. By using the `find(...)` function, one can look for an item in cache. First, the DRAM cache, using a hash table to keep track of its entries, looks for the item. If the item is not in DRAM, navy looks for it in the LOC. If the item is not in LOC, navy looks for it in SOC. If the item is in flash, navy insert it to DRAM, which then returns its `itemHandle`. If the item is also not in the flash cache, navy returns an empty `itemHandle`. 2.3 illustrates that workflow.

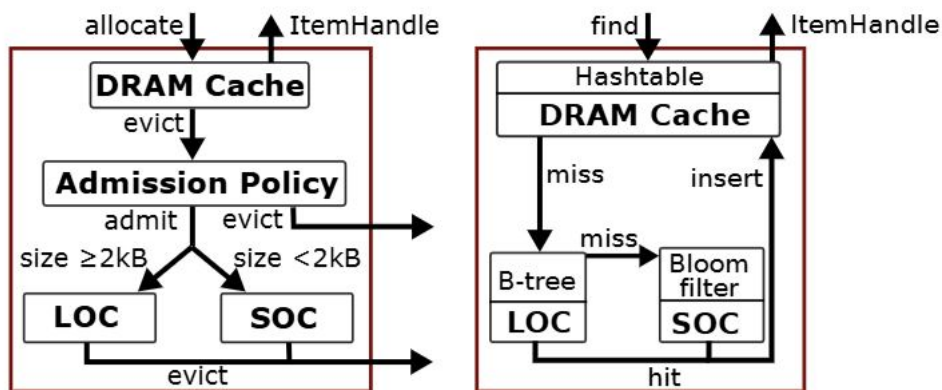


Figure 2.3: The allocate and find framework for navy [1]

2.8 Machine learning based caching

In recent years, using machine learning for caching has become a popular approach to improve performance. Researchers used various methods to improve several different metrics, most notably including reinforcement learning, supervised learning, and hybrid approaches that combine both heuristic and traditional methods with machine learning techniques.

2.8.1 Reinforcement learning

Reinforcement learning itself is a theory of an agent interacting with an environment and learning from this environment to make optimal decisions. With recurring feedback, the agent will steer towards an optimal behaviour in this environment. Using reinforcement learning for caching uses the cache itself as the environment and policies as the agent learning optimal behaviour for caching. Vietri et al., for example, proposed LeCaR. LeCaR employs two eviction policies, LRU and LFU, and switches between them, effectively learning the optimal probability distribution for every state through regret minimization. When cache space is highly constrained, LeCaR achieves an up to 18 times improvement over ARC [22].

Another RL-based approach comes from Rodriguez et al., introducing CACHEUS. CACHEUS categorizes workloads into four different types: LRU friendly, LFU friendly, scans, and churn. The CACHEUS algorithm is an adaptive cache replacement policy, but it is different from LeCaR, because it has no fixed hyperparameters. CACHEUS then uses two experts, CR-LFU and SR-LRU, achieving competitive results across a majority of workload and cache-size combinations compared to state-of-the-art methods [17].

2.8.2 Supervised learning

Supervised learning itself learns from labeled data, allowing the algorithm to learn towards correctness and pattern recognition. For caching, this is often more so the case for imitating an optimal policy. For example, Belady's algorithm, which is an optimal eviction policy, always evicts the item not used for the longest time in the future. This policy is not achievable, however, supervised learning algorithms are learning from it. Song et al introduced Learning Relaxed Belady (LRB). LRB uses regression to predict an object's time to next access. If an object's time to next access exceeds a certain boundary, called the Belady boundary, then the object can be evicted. For production CDN traces, LRB reduces byte miss ratio by 4-25% compared to heuristic methods [20].

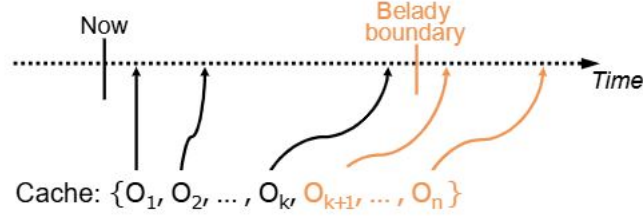


Figure 2.4: The relaxed belady algorithm, items $O_{k+1}, \dots, O_n$ are beyond the threshold [20].

2.8.3 Hybrid approach

Heuristic aided learning prefetching (HALP) is an example of mixing traditional policies with learned methods. HALP uses LRU to identify eviction candidates and then employs a neural network to predict the best eviction candidate. For CDN traces, HALP reduces byte miss ratio by 9% [21]. This is a typical method visible in hybrid approaches, using heuristic methods to do the heavy lifting and acting as filters, and then employing more complex and learned methods to improve the performance, specifically for those cases where heuristic methods fail. Later in the related works section, we will provide more detailed information about some prominent approaches.

2.8.4 Challenges of ML in caching

Feature selection

Learning is always dependent on the available features of objects. However, there exist many features, and choosing the right feature can be a difficult task. Features can include time to next access, access count, object size, and more. More complex features might also exist, such as ones showing a correlation between objects. Therefore, feature selection is a crucial task. Trying to learn high-dimensional features can induce a significant overhead, while learning low-dimensional features might not improve the performance at all, while still inducing a small overhead. LRB, for example, only aims to learn about the time to next access, making it simple while still providing meaningful results. Additionally, generalization across workloads or even use cases is difficult, as different workloads have different features, and different use cases might need an optimization of other features [20].

Overhead

Since ML solutions require more information than traditional methods, and often work in combination with them, they also induce an overhead. This exposes a new problem, the

resulting trade-off between the increase in performance and additional overhead. For an increase in performance, some overhead is usually acceptable, but when the overhead becomes too large, it may offset any performance enhancement. For this reason, researchers steered away from using ML for every decision and using ML only on as many decisions as needed. The solutions we explored have already proven this by inducing only small overheads while still noticeably increasing performance [1].

2.9 Related work

As we have seen, previous studies use ML to improve caching performance and efficiency. As such, research applied ML towards caching to solve for various optimization and efficiency problems, including reinforcement learning, supervised learning, and hybrid methods.

2.9.1 Baleen

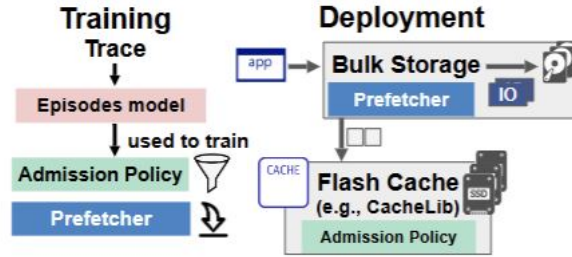


Figure 2.5: The architecture of Baleen [23].

Because flash caches have limited write endurance, caches need to reduce the workload. Wong et al. implemented Baleen on top of the cachelib engine to reduce backend workload using machine learning. The model predicts whether a data block is to be cached and is soon to be accessed again, and if it is above the set threshold, then it is admitted and discarded otherwise. Predictions are made based on static and dynamic features of the data. Baleen uses Wong et al. (GBM) to make its decisions. The model trains by imitating a theoretical optimal policy, OPT. Baleen generates episodes from traces, which are a group of accesses to a data block during the time it would stay in the cache, and labels them. Moreover, Baleen uses regression models to predict what segments to prefetch and whether prefetching this segment is worth it. First, two models are used to predict the start and end of the segment range. These models furthermore imitate the OPT. To determine the worth of prefetching the selected episode, a new model is used that calculates the expected benefit in comparison to the cost and decides

whether to prefetch the episode if the difference exceeds the threshold. The result is a 12% reduction in peak disk load and a 17% reduction in total cost of ownership [23].

2.9.2 MAT

Yang et al. introduced Machine Learning at the Tail (MAT), which uses a machine learning model on eviction candidates and is implemented on top of cachelib. While machine learning reduces cache miss ratios, state-of-the-art models need to make many predictions. MAT’s goal is to reduce predictions made by the ML model while maintaining a comparable miss ratio. If eviction is necessary, the priority queue pops the item at the tail, and MAT estimates the likelihood that the item to be evicted will be used again by computing its expected time-to-next-access (TTA). If this likelihood is higher than a set threshold, the item remains in cache, and MAT checks the next item from the possible candidates. MAT trains the model using the set of recent eviction candidates, for which it calculates its true TTA label. MAT uses Gradient Boosted Decision Trees (GBDT) by default [24]. To evaluate the system, MAT uses four CDN, two in-memory, and two storage traces.

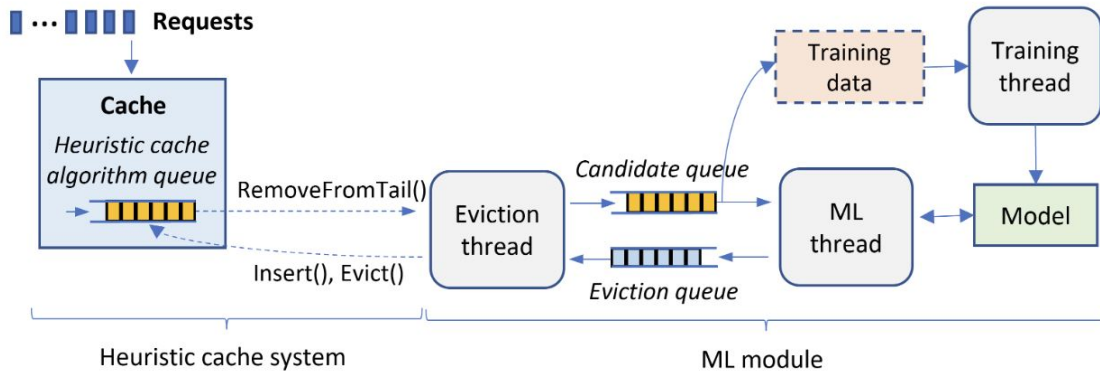


Figure 2.6: The architecture and data flow of the MAT framework [24].

2.9.3 GL-Cache

Traditional caching systems fail to differentiate efficiently between objects. To tackle this, Yang et al. introduced GL-Cache (group-level learning), focusing on the eviction of items. When items are admitted to cache, they are assigned to a group (grouping by write time) and stay there for their lifetime in cache. When eviction is needed, a two-level eviction strategy is employed. In the first level, groups are ranked and assigned a predicted utility value, and then the group with the worst ranking is chosen. Then this group is merged with adjacent groups,

and only the top items are retained, and the rest are discarded. Retaining objects is based on recency and size. Group-level learning uses GBM and is trained on group-level features. The GL-Cache uses online collected data of features and utility of object groups to train a model, and new training data is sampled by cached object groups. The cache was evaluated using block I/O traces from the CloudPhysics Dataset and Microsoft Research Cambridge [25].

2.9.4 Raven

Caching algorithms use simple heuristics, but these heuristics fall short for complex patterns or workloads changing over time. Leveraging the Belady algorithm, Berger et al. introduced the Raven framework. Raven uses a Density Estimation Network trained on a window of past requests and produces an estimated arrival time distribution for each object. When the cache needs to evict an item, it queries the MDN to predict a distribution for each object candidate. Then it uses a Belady-like algorithm approach by evicting the item that will arrive farthest in the future. Furthermore, the framework can be used for optimizing object hit ratio or byte hit ratio, depending on the objective given. To effectively learn the arrival process, a Mixture Density Network is employed that can also learn arbitrary arrival processes, unlike simpler models. Moreover, considering the nonstationarity of arrival processes, a recurrent neural network (RNN) extracts temporal dependencies by processing each object's history between requests and producing a history embedding [9].

Chapter 3

Design and Implementation

The codebase can be found on out GitHub¹.

3.1 Architecture overview

3.1.1 Cache Architecture

Two-level cache We set up a two-level caching system using the Navy library from Cachelib, consisting of a small DRAM cache for the first level and a larger SSD cache for the second level. If a write occurs, we call the `insertOrReplace` function from Navy, which either inserts the item into DRAM or updates it. If the DRAM cache is full, items must be evicted according to the chosen eviction policy. After DRAM eviction, the admission policy decides to insert the item into the SSD or reject it. Upon rejection, the cache works as a write-back cache, writing the data to the HDD. However, if the item is accepted, we only insert into SSD; no HDD write occurs. A read is looked up in the cache; if the item exists, it is fetched from the cache. Otherwise, if the item is not cached, it is now inserted into the cache.

Write back We implemented a write-back behaviour. An update replaces the current data, and only upon eviction data may be written to the SSD; however, this decision is subject to the underlying admission policy. Navy automatically manages the process of inquiring the admission policy and writing to SSD, but if the item is already present in SSD (indicated by `isNvmClean`), then no additional write occurs, improving performance and reducing SSD writes [1].

¹<https://github.com/ViperX88/Investigating-ML-Based-Solutions-for-SSD-Caching>

3.1.2 Trace Replay Architecture

Block operations

We use a fixed block size of 4 KiB, aligning with the operating system granularity. Trace operations are therefore replayed at block granularity. For multi-block requests, we decompose the request into sub-block requests and iterate over blocks by computing:

$$b = \left\lfloor \frac{\text{offset}}{4096} \right\rfloor \cdot 4096, \quad b+ = 4096 \text{ until } b \geq \text{offset} + \text{size}$$

$$r_{\text{Start}} = \max(b, \text{offset}), \quad r_{\text{End}} = \min(b + 4096, \text{offset} + \text{size}), \quad \text{len} = r_{\text{End}} - r_{\text{Start}}$$

with ℓ sub-range length; and Δ the offset inside the 4 KiB block.

The computation is valid for both read and write requests.

Reads and Writes

For each block, we do a lookup on cache `find`. Upon a hit, the data is served from memory and cached, but when the data is in SSD, it is promoted to DRAM. Upon a miss, the data is fetched from the origin, memory is allocated in the cache using `allocate`, and then the data is inserted using `insert`. Since caching happens at block granularity, the entire block is cached. If the block is already present in cache, it is marked dirty and updated. If the block is not present, we allocate a full block in the cache and insert the payload into it. Otherwise the cache behaves same as subsection 2.7.4. For evicted items from DRAM that are not admitted to SSD, and items evicted from SSD, we manually write the data to HDD, since navy does not have any access and cannot issue I/O itself.

Direct I/O

To better test our cache, we use `O_DIRECT` to open the origin file. By using `O_DIRECT`, we disable the Linux page cache, yielding more accurate results since operations are not cached a second time. Our traces consist of disk accesses, therefore not cached at the main memory level. Through `O_DIRECT`, we can, however, achieve direct I/O. We configure navy to use direct I/O:

Algorithm 1 Navy Config

```
1: nvmConfig.navyConfig.setSimpleFile(kNvmFile, kNvmBytes, /*useDirectIO*/
   true);
```

However, direct I/O requires 4 KiB alignment. Therefore, we allocate I/O buffers with `posix_memalign`.

3.1.3 Cachelib

Cache Initialization

First, we need to initialize the cache. In order to do that, we create one `CacheAllocator` instance. We then set the DRAM capacity and SSD capacity. Furthermore, we need to decide on an eviction policy for the DRAM cache. In our case, we switch between the two policies:

Algorithm 2 Cachelib Allocator

```
1: using Cache = facebook::cachelib::LruAllocator;  
2: using Cache = facebook::...::TinyLFUCacheTrait>;
```

Algorithm 3 Flash cache

```
1: Cache::NvmCacheConfig nvmConfig;
```

We create the flash cache through 3.1.3, which is backed by a single file on SSD and opened with `O_DIRECT` to bypass the Linux page cache. We use one pool and one allocation class aligned to our block granularity (4 KiB). Navy’s standard admission policy for SSD is always-admit.

Cache operations

Using Cachelib’s API, we use several of its operations, including:

1. `cache.find(key)` returns a handle on hit and is empty on a miss. If the key object is in flash, Cachelib returns its item handle and inserts the item into DRAM.
2. `cache.allocate(poolId, key, size)` returns a buffer that can be written to. The data can be written to it, after which we call `cache.insert(handle)` to insert the data into the cache.

Shutdown

We run a single-threaded replay. Navy runs asynchronous I/O, and we flush at the end of the trace using `fdatasync` on the origin file to destroy the cache and to flush any remaining I/O operations.

3.2 Baleen

We use Baleen as a machine learning based admission policy to compare its performance to always-admit and reject-first. Baleen inquires from a trained model an item's score, determining admission of the item. The goal is to avoid one-time hits, decreasing SSD writes and giving more space to multi-hit objects. Baleen also features machine learning-guided prefetching, but we do not test for that.

Features

In order for Baleen to predict items' future accesses and thus make admission decisions, it needs features. Thus, Baleen creates an 18-dimensional feature vector, including static data and access history. For our traces, we can summarize this vector into:

1. **Operation Type** encodes if the operation is a read or write.
2. **Metadata** such as namespaces and user identifiers.
3. **Size and Offset** encoding the logical offset (end and start) as well as the size of the request.
4. **Dynamic access history** divided into two parts:
 - **Block recency** including six counters tracking how often an object was accessed during the last six windows. Because this tracking is based on blocks, it represents the temporal recency.
 - **Segment coverage** also includes six counters tracking how many sub-segments were accessed during the last six windows. The tracking is based on segments and represents high-level tracking or spatial locality.

The features are tracked in a map `ObjectInfo[key]`. If an object is accessed, the map is updated accordingly. The model to admit is a binary classifier that predicts whether future access is likely, and thus if admission makes sense. The model is trained offline, at runtime only inference happens. For the history features, we track how often the given object was accessed during the last 6-hour interval (represented by the six vectors). The segment coverage specifically counts how many distinct sub-segments were accessed during the interval. At runtime, when an access happens, the current hours counter is incremented and the new last access is recorded, computing both recency and frequency. Note that we use a simple map to track these features. In production, Baleen uses a

count-min sketch to track these features. However, since we run single-threaded, a map is sufficient to store this information.

Table 3.1: Feature_vector.

Feature	Description
feat_metadata op	Request type (READ=0, WRITE=1)
feat_metadata ns	Timestamp (nanoseconds)
feat_metadata user	Identifier (integer)
feat_metadata_size start	Start 4 KiB sub-block index
feat_metadata_size end	End 4 KiB sub-block index
feat_metadata_size size	Request size
feat_dynamic_b 0..5	Temporal dynamic access counters
feat_dynamic_c_combined 0..5	Spatial features

Admission control

After creating the feature vector, we pass it to the decision model, a LightGBM decision tree, that outputs a score $s \in [0, 1]$. This score indicates the level of confidence of the model that the admission of the object yields an improvement. The higher s is, the more confident the model becomes. Furthermore, we set a threshold t . If the confidence of the model surpasses t , we admit the object. Otherwise, we reject it. We extract the threshold after training. We can then formulate the admission decision function as:

$$f_t(s) = \begin{cases} \text{admit}, & s \geq t \\ \text{reject}, & s < t \end{cases}$$

In production, Baleen adjusts t to control SSD writes, tuning the cache’s write load. In our code, we do not update t as we do not set a target write rate.

Granularity

Wong et al. initially created Baleen to be trained and tested on Meta traces. Therefore, Baleen works on segments of 2 MiB granularity. Since Baleen’s training happened at this granularity, we preserve the size when testing Baleen. Each segment has a 4 KiB bitmap tracking modified blocks. On a hit, we fill the rest of the segment.

3.2.1 Cachelib integration

Cachelib allows for the modification of admission policies. We use the default admission policy of Cachelib (always-admit) and then apply our baleen policy immediately afterward. This way, Baleen acts as the real admission policy. For a read miss, we create the feature vector and inquire the model about the score of this segment. If the score is above the threshold, we admit the item to the cache. From then on, insertion is the same as before. After that we continue to track the items accesses during its residency in cache. On a miss, we do the same as before, simply writing the item to the origin and not to the cache. The machine learning predictor does add overhead. However, a manageable one.

3.3 MAT

Like Baleen, we implemented MAT on top of Cachelib. We use the same Architecture as before, but similar to Baleen, we changed the policy, and in this case, the eviction policy. MAT runs using the default LRU eviction policy, which we overwrite at runtime by setting `useEvictionControl = true`. This is a function special to the author’s Cachelib fork (https://github.com/yangdsh/Cachelib_2023) and enables the MAT configuration. Specifically, upon eviction of an item, the machine learning model is queried and may overwrite the decision and extend the item’s time in cache. MAT is an eviction policy augmenting the standard LRU policy.

3.3.1 Features

We implemented MAT like we implemented our baseline policy, using write-back and a 4 KiB block size. For each cached object, we create a metadata feature vector upon which the model can make decisions. For that, we use Cachelib’s `ObjectInfo` struct, which includes a 32-bit bitmap and age counters. On an access, we update the feature vector of the object. The bitmap acts as a sliding window for recent accesses, while the counters measure the time since the last access. We then use this structure to compute an object’s time-to-next-access (TTA), upon which the machine learning model makes its prediction.

3.3.2 Read

Upon a read, MAT first looks for the key in the cache. If we find the key, it counts as a hit, and we refresh the object’s features (using `ObjectInfo`). We then call the `insertOrReplace` function to reinsert the object into the cache, effectively overwriting its previous entry and

moving the item to the head of the queue with new metadata. Nevertheless, if we do not find the key, MAT computes features for the new object, which we fetch from the HDD. We generate a new cache entry using `cache->allocate(...)` on the objects' freshly generated metadata.

3.3.3 Write

MAT generates fresh metadata for the object. Then, we allocate space in the cache and insert the item. Because we use write back, if the object is updated while residing in cache, we mark it dirty and write the data back to HDD later. We keep track of the dirty objects using another set. But when moving an item from DRAM to NVM, we do not write its data back to the origin.

3.3.4 ML Model

When the code starts, we pass a YAML file that integrates the machine learning part. The YAML file points to the trained model, a text file, which is called by MATLAB during runtime using the model's parameters. The model is a LightGBM model, a regression tree. We only call the model upon an eviction, where we choose a set of candidates from the tail of the queue. We do this using plain LRU, acting as a filter for the model. The model predicts the TTA for every candidate, and if the TTA is above a set threshold, we evict the candidate. Otherwise, the model is not confident enough, and we do not reinsert it into the cache. If none of the candidates are suitable for eviction, the eviction queue is empty, and we evict the next item at the tail of the LRU queue. Through this approach, MAT uses heuristic algorithms as filters and only invokes machine learning on those filtered decisions, reducing the machine learning overhead. MAT implemented its own eviction controller through which we called the LightGBM model. This function is not part of the standard Cachelib library and is special to the fork from MAT.

Chapter 4

Results

4.1 Methodology

4.1.1 Cloudlab

Cloudlab is an online provider of resources, allowing researchers access to bare metal testbeds to run their experiments. For our experiment, we used the c220g2 node from the Wisconsin cluster with Intel Xeon Processor E5-2660 v3, 160 GB DRAM, 440 GB SSD, and 2x 1.1 TB HDDs [5].

Table 4.1: CloudLab testbed summary (Wisconsin, c220g2).

Site	CloudLab (Wisc), c220g2
CPU	2× Intel Xeon E5-2660 v3 (Haswell), 10 cores/socket (20 cores, 40 threads)
Base frequency	2.60 GHz
Memory	160 GB ECC DRAM
SSD	480 GB SATA SSD
HDD	2x 1.1 TiB 10K RPM 6 6G SAS SFF
NIC	Dual-port Intel X520-DA2 10 Gb NIC
OS	Ubuntu Linux 5.15.0-86-generic
Filesystem	ext4

4.1.2 Workloads

To evaluate our results, we use the MSR Cambridge block-level storage traces. These traces include:

1. **U_{sr}_0**: user home directories trace.

2. **Rsrch_0**: research data server trace.
3. **Prn_0**: print server trace.
4. **Prxy_0**: enterprise web proxy trace.
5. **Mds_0**: media server trace.
6. **Ts_0**: terminal server trace.

Each trace is a sequence of operations designed to simulate real-world behavior in content systems, containing requests (reads) and writes to the system. These traces also differ in duration, size, and number of requests, resulting in varying workloads. For instance, workloads such as **Prxy_0** have a high temporal locality, small working set, and frequent reuse, while others like **Usr_0** exhibit larger working sets. An overview of the traces can be seen in Table 2. Because traces can have durations of several days, we replay traces without following the timestamps and instead handle requests instantly. Through this approach, we save significant time when testing. When using a machine learning algorithm, we train the algorithm on a subset of the trace. In the case of Baleen, we train the model on 10% of the trace. We use the starting segment to tune the parameters, and use the rest for testing.[13].

Table 4.2: MSR Cambridge traces

Trace	Size	Reads	Writes
Usr_0	48.38 GB	9,287,751 (72.15%)	3,585,523 (27.85%)
Rsrch_0	12.81 GB	365,113 (11.22%)	2,889,165 (88.78%)
Prn_0	59.12 GB	3,490,810 (19.79%)	14,144,956 (80.21%)
Prxy_0	56.85 GB	817,125 (3.61%)	21,825,425 (96.39%)
Mds_0	10.62 GB	863,725 (29.61%)	2,052,937 (70.39%)
Ts_0	15.47 GB	1,091,561 (25.89%)	3,124,896 (74.11%)

4.1.3 Caching Policies

We compare four different policies. All tests are conducted using two-level caches in Cachelib.

1. **LRU + always-admit**: This is our baseline policy. LRU is the standard eviction policy in cachelib. In Cachelib, admission is dependent on a variable p , with $p \in [0,1]$. If $p=1$, the cache always-admits. If $p=0$, the cache always rejects. Not setting the p specifically results in the standard policy, which is always-admit. This policy is for the SSD cache. DRAM does not have an admission policy. LRU evicts the least recently used item.
2. **TinyLFU + always-admit**: Our admission policy is always-admit like before, but we change the eviction policy to TinyLFU.

3. LRU + **reject-first**: Our none machine learning based admission policy. reject-first rejects items the first time, but if during a certain time period these items are accessed again, they are admitted. For that, reject-first has several parameters, which we set to:

Algorithm 4 Reject-first

```
config.setNvmCacheAdmissionPolicy( std::make_shared<facebook::cachelib::RejectFirstAP<Cache>(
/*numEntries*/ 1000000, /*numSplits*/ 4, /*suffixIgnoreLen*/ 0, /*useDramHitSignal*/
true));
```

These values align with the values of the traces and the system.

4. TinyLFU + **reject-first**: The same as before, we now use TinyLFU with the same admission policy.

We test each combination of policies under the same conditions: 128 MB of DRAM, 100GB of SSD, 4 KiB Block size, and write-back upon eviction.

4.1.4 Baleen training

We trained the Baleen model using the official simulator from (<https://github.com/wonglkd/Baleen-FAST24>). The training was offline and on the same traces we used for replay.

Data preparation

Baleen uses traces from [14]. These traces have different headers than the MSR Cambridge traces used in our research. Therefore, we converted our traces accordingly. First, the MSR traces contain timestamps, but Baleen expects floating-point numbers, so we convert the timestamps to seconds. Baleen has different operation codes than before; for a read, Baleen uses GET_PERM, and for writes, PUT_PERM. These are the only ones because the MSR traces only contain these two. Furthermore, Baleen uses 8 MiB segments. We compute requests from lines from the trace to segments, which include the segment identifier, the offset, and the length of the request. A request can include several segments. This way, we preserve the granularity of Baleen needed for training. We sample 10% of the trace for training. We create a sliding window that runs for the first 10% of the trace. Baleen expects four files for training. A trace file, representing the trace and consisting of trace entries. A keys file, which is a per-segment count of total accesses. A header file, which describes the columns. And lastly, a checksum for integrity reasons.

Training

We set the following parameters for training, following the official default guideline from Baleen:

- **Sample ratio** We set the sample ratio to 0.1, using 10% of the trace to train the model.
- **Training window** We set the training window to max= 86400 seconds, using at maximum the first 24 hours of the trace for training. This is consistent with baleens in production training.
- **Eviction age** Eviction age defines an item’s residency in cache, specifically how long the item should reside in cache before being evicted. We set it to 5892.856, consistent with Baleen’s default value.
- **Feature Set** The features the model was trained on can be found here ??.
- **Write rate** We trained the model on the default target write rate of 35.999. However, we provided a range of target write rates to determine the one that yields the best results. We do not use that target write rate during runtime.

Learning

We trained the machine learning model using LightGBM, a library for gradient boosted decision trees. We mimicked the production training of Baleen, using 500 trees with a maximum of 63 leaves. Admission is based on a binary classifier, which learns the optimal probability for admission. We completed training on one CPU on the Cloudlab node, and it took around 20-25 seconds. We repeated this process for every trace we tested.

Output

Upon finishing training, Baleen produces an analysis CSV and the saved model. Using treelite, we compiled the model into a C++ library that can be called at runtime. Furthermore, we collected the feature names used and a JSON containing the cutoff and segment size [3].

4.1.5 MAT training

We performed all operations on CloudLab and only used its CPU. We reused most parts of the code used for the baseline. Since MAT has a Cachelib fork, we called the fork’s APIs. We have implemented a 2-pass approach for MAT, one for training the model and one for testing. MAT trains by computing objects TTA using the objects’ feature vector. On the

training pass, we write a CSV file of the computed features, creating a dataset of features to train our model on. We train a LightGBM regression model offline, using parameters similar to the ones used by MAT, including a regression L2 objective, 32 trees with maximum depth 32, and a learning rate of 0.05.

4.1.6 Evaluation metrics

To assess the different policies, we test test cache on the following metrics:

Cache hit ratio

Cache hit ratio refers to the fraction of requests served from the cache instead of memory. It is often measured in terms of object hit ratio (fraction of hit accesses) or byte hit ratio (fraction of hit bytes). A higher hit ratio translates to lower backend load, a key improvement of tasks [2].

Latency and Response Time

Latency refers to the time between requesting data and receiving that data, measuring the response time for read requests. Typically, caches should have a significantly lower latency than using plain disks, as the cache should serve the bulk of requests, and as the cache provides data faster, this should speed up the process. Lower latency stems from caches being more effective and able to provide most of the requests. However, a challenge remains for cache misses, when data is requested and searched for in the cache (cache lookup), but not found, it should not add a significant overhead. In our hybrid cache, we measure DRAM hits, SSD hits, and cache misses that go to the origin. Furthermore, the time needed to finish the trace is measured as response time. Usually, a cache writing more to SSD should achieve a faster response time [1].

Write amplification

For caches, especially those with limited lifetimes such as flash, measuring the amount of data written to the device over time is important. Total writes per day are often measured, with excessive write rates wearing out devices. Write amplification then happens when more data is written than the size of that data actually is, often due to eviction or additional metadata [23].

Flash endurance

Since flash only has a limited lifetime, measuring the impact of policies and respective write rates can give insights into their impact. A common approach is to measure the write rates of policies, followed by measuring the expected lifetime under each policy. An example of this is Flashield, an admission policy, where the authors measured the reduction in write rates to flash while keeping hit rates [1].

Disk head time

Similar to flash endurance, disk head time measures how much is written to the disk over a certain time period. This is most common for SSD caching, where the SSD acts as a caching layer for the underlying HDD. The measurement is then to see how busy the disk was over that time, with successful caches often reducing disk head time. A famous metric often used is peak disk head time, referring to the longest time the disk is busy, and many researchers try to specifically lower peak disk head time [23].

4.2 Results

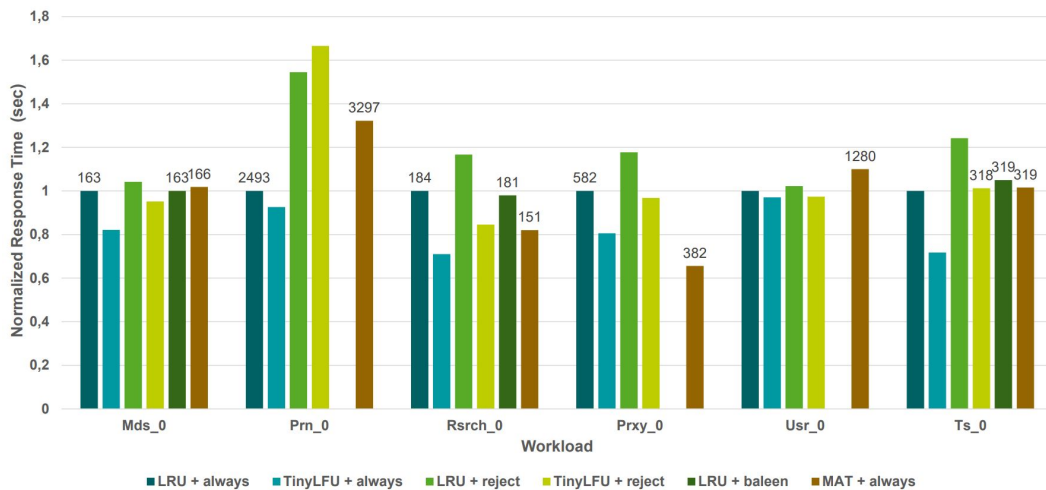


Figure 4.1: Response Time normalized to LRU + always-admit.

We normalized results for response time to LRU with always-admit, which serves as our baseline. We measure in seconds. We can observe that TinyLFU outperforms LRU across workloads. This trend persists no matter the paired admission policy. Moreover, using reject-first results in worse response time. That difference can be noticeable, like in Prn_0, or small, like for Usr_0. Prn_0 is the only case in which TinyLFU performs worse than LRU, but also

only when paired with reject-first. Using ML-based solutions can improve or worsen overall response time. For example, MAT is faster for Rsrch_0 and Prxy_0, and slower for Prn_0 and Usrc_0, while achieving the same response time for Mds_0 and Ts_0. For Baleen, we observe an improvement for Rsrch_0 and a worse response time for Ts_0. For Ts_0, Baleen achieves the same response time as Mds_0. However, for Baleen, we do not report results for Prn_0, Prxy_0, and Usrc_0, as testing failed.

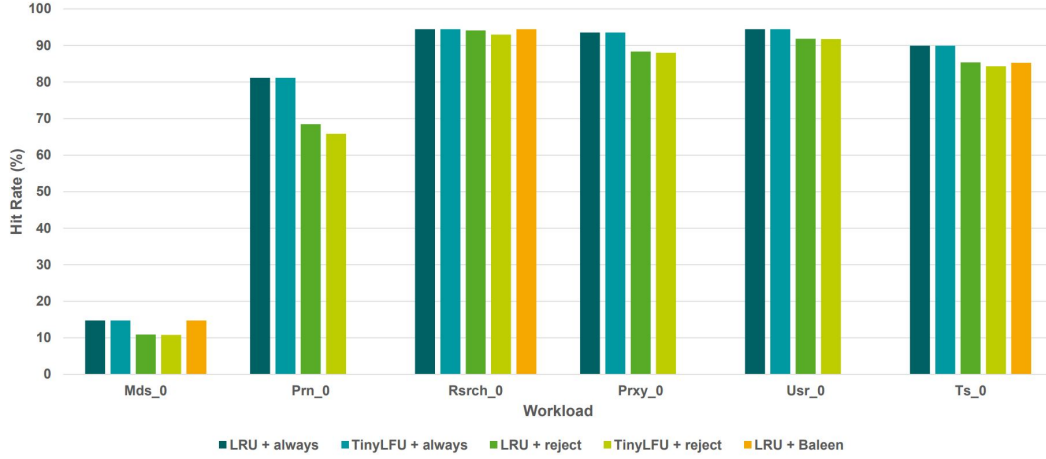


Figure 4.2: Hit Rate.

We report hit rate in % and normalise the hit rate to LRU with always-admit. When we pair LRU and TinyLFU with always-admit, both achieve the same hit rate. When paired with reject-first, hit rates are similar, but LRU achieves a slightly higher hit rate. Baleen achieves the same hit rate as the baseline for Mds_0 and Rsrch_0, but is slightly worse for Ts_0. We report the hit rate for MAT separately.



Figure 4.3: SSD Writes normalized to LRU + always-admit.

We measure SSD writes in MB, and normalize results to LRU with always admit. TinyLFU

across workloads writes less to SSD than LRU when paired with the same admission policy. Differences can be small, like in Mds_0, or more significant, like in Prxy_0. Using reject-first results in much fewer writes to SSD compared to always-admit. MAT achieves similar results to the baseline. However, for Mds_0, MAT writes slightly less to SSD, while for Rsrch_0 and Usr_0, it writes slightly more. Using Baleen results in fewer SSD writes in general, with results similar to the baseline for Mds_0 and Ts_0, or a more noticeable difference for Rsrch_0.

Across workloads, TinyLFU achieves better response time than LRU while maintaining the same hit rate and writing less to SSD if both use the same admission policy. That is because TinyLFU keeps frequently accessed data in cache longer compared to LRU, which evicts more often. Therefore, accesses hit the first-level cache more often for TinyLFU, reducing its response time compared to LRU.

Always-admit allows all data to be admitted to flash. As such, it comes as no surprise that always-admit yields higher SSD writes than reject-first. In comparison, reject-first writes on average 37,5% less to SSD than always-admit across different admission policies. However, reject-first takes around 24,7% longer and reduces hit rate by 10%.

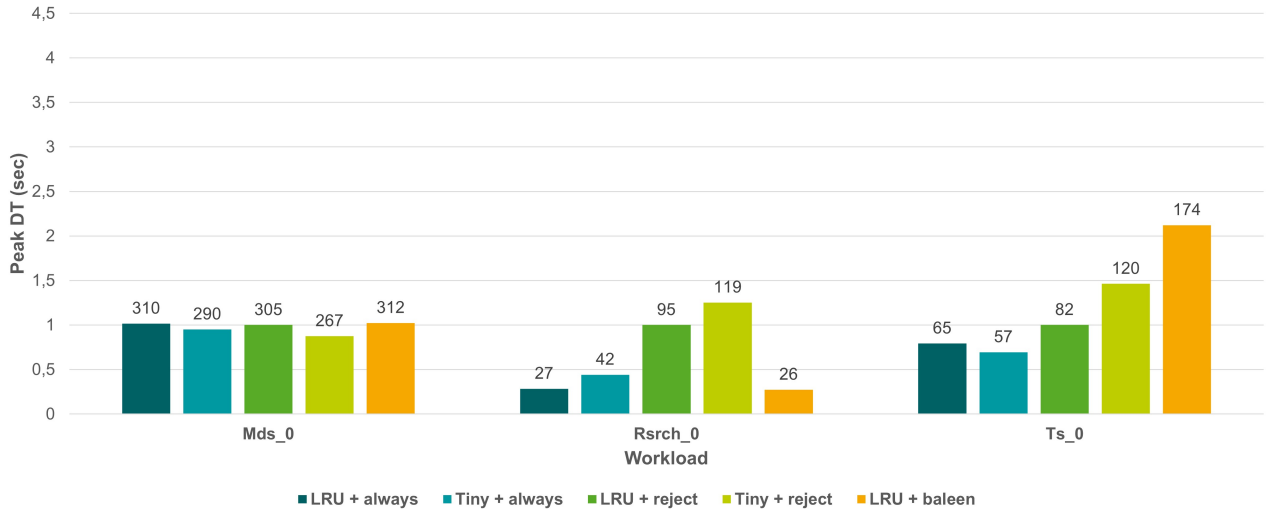


Figure 4.4: Peak DT normalized to LRU + reject-first.

For peak DT, we normalized our results to LRU with reject-first and reported the results in MB. That is because Yang et al. in their work tested against a variant of reject-first. Baleen behaves similarly to always-admit, however, with a few differences. Baleen’s overall response time is almost identical to that of LRU with always-admit. Moreover, we see a similar hit rate,

with Baleen decreasing the hit rate slightly. Baleen writes less to SSD, reducing SSD writes by 6%. Because of its similar behavior to LRU with always-admit, Baleen also achieves a better hit rate and response time compared to LRU with reject-first. However, compared to TinyLFU with always-admit, Baleen’s performance degrades. For Rsrch_0, Baleen achieves its best results, reducing peak DT by 73% compared to using reject-first. Nevertheless, Baleens increase peak DT by 2% for Mds_0 and by 120% for Ts_0.

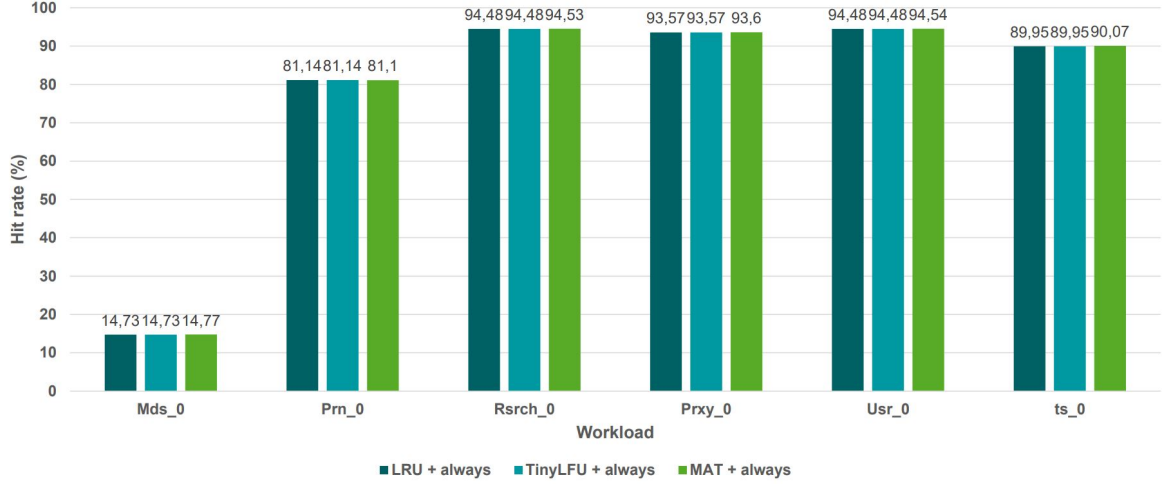


Figure 4.5: Hit Rate.

We report a secondary hit rate figure, showcasing the hit rate for MAT with always-admit and compare it to LRU with always-admit and TinyLFU with always-admit, because due to time reasons we were not able to test MAT with another admission policy than always-admit. We report these results separately to better visualize the small but important changes in results. For all workloads but Prn_0, MAT improves hit rate on average by 0,05%, for Prn_0, MAT reduces hit rate by 0,04%. Otherwise, MAT behaves differently depending on the workload. For Mds_0 and Ts_0, MAT achieves similar results to LRU with always-admit. For Prn_0 and Usr_0, MAT has a longer response time and writes slightly more to SSD. For Prxy_0 and Rsrch_0, MAT is about 25% faster. However, we can also observe an increase in SSD writes by 7%.

4.3 Discussion

4.3.1 Heuristic methods

We can clearly observe an improvement in results across workloads for TinyLFU compared to LRU, regardless of the paired admission policy. For always-admit, TinyLFU achieves a better response time and writes less to SSD, all while achieving the same hit rate. For reject-first, the results are similar. TinyLFU achieves a faster response time for all workloads but Prn_0, where TinyLFU takes longer. Moreover, the hit rate difference is small, only for Prn_0 LRU achieves a 3% better hit rate, explaining the difference in response time. Otherwise, the differences are small in favour of LRU. The reason for TinyLFU’s superior performance is a result from its ability to keep hot data in the cache for longer periods. TinyLFU in Cachelib is a modified version of LRU. These results are in line with the expectation [6]. Furthermore, reject-first performs much worse in terms of hit rate and response time compared to always-admit, but also writes significantly less to SSD. This showcases the trade-off systems nowadays face, choosing between better performance or longer lifetime (due to fewer writes to SSD).

4.3.2 Baleen

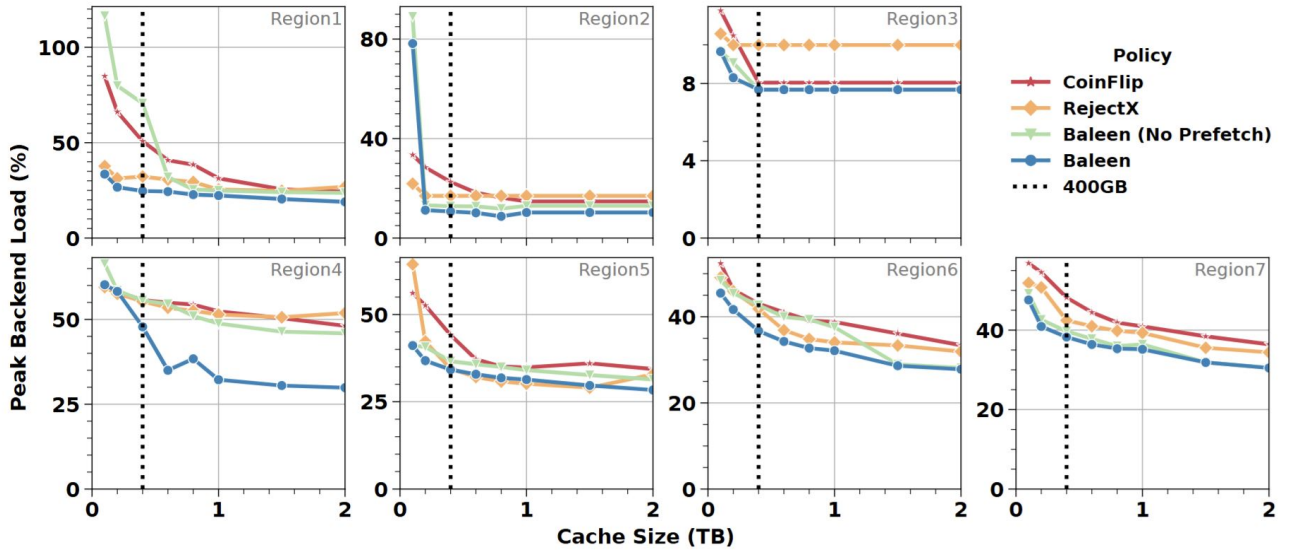


Figure 4.6: Baleen's Performance to Cache Size [23].

Baleen showcases a high workload sensitivity, with results varying from clear improvements to worsening performance. Partial support for Baleen can still be seen, which implies that, depending on the workload, Baleen can improve performance. However, Wong et al. tested

using much bigger caches and also tested for different workloads, which explains the possible worsening of performance. Moreover, Baleen is tested and trained on workloads from Tectonic, which are limited in size. That explains the model’s failure for large workloads. We did not get any results for traces of sizes around 50 GB, showcasing Baleen’s lack of adaptation for bigger size workloads. In conclusion, we can still see some promising results, with Baleen reducing peak-DT as well as lowering writes to SSD. However, we can also observe the opposite. As can be seen in 4.6, Wong et al. state that using a cache size < 400 GB can result in Baleen worsening peak-DT, therefore, our results are not too surprising.

4.3.3 MAT

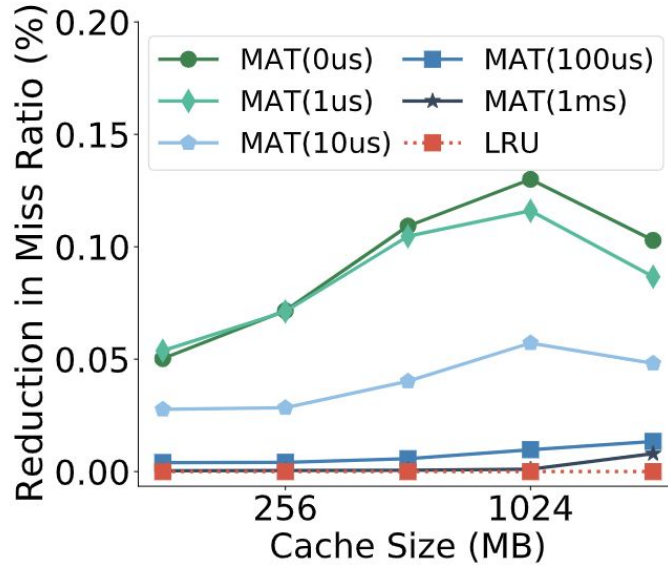


Figure 4.7: Reduction of Miss Rate using MAT [24].

Our results show that while MAT can differ in response time and SSD writes, it almost consistently outperforms heuristic methods when it comes to hit rate, yet with only minor improvements. The results exactly mirror the expectation of Yang et al., who expect a 0,05% miss rate reduction (conversely, hit rate improvement) when using 128 MB DRAM 4.7. Even though the improvement is negligible, the results are as expected, implying that for bigger caches, we can expect better performance gains by MAT. Otherwise, MAT behaves similarly to LRU. This behaviour can be explained by MAT being an extension of LRU, with LRU acting as a filter, and MAT only making the final eviction decision. Also, in case the ML threads are too slow, MAT falls back to plain LRU. As already stated, MAT performs differently for response time and SSD writes. A slight increase in response time is in line with expectations

from Yang et al, and similarly, MAT can also be faster. This depends on the incurring ML overhead, which can both improve and degrade performance. As for writes to SSD, we observe MAT only some times writing more to SSD, but only with a small difference. Otherwise, MAT behaves similarly to LRU with always-admit.

4.4 Future work

Research on ML-based solutions has yet to be conducted in a broader range and has to be updated continuously as systems and workloads become more complex over time. Furthermore, a vast amount of caching systems and ML-based solutions exist, however, the results remain fragile and often lack a deeper and more comprehensive work, mirrored by the missing robustness and great variation of results. Moreover, nowadays most production caching systems still use heuristic methods, which shows a lack of adaptability in these solutions. Evaluation often focuses on single metrics, but previous studies have shown how such results fail to account for other metrics, resulting in an overall worsening of performance. Research often revolves around improving performance, but the cost is often disregarded. Improving performance is offset by additional incurred costs. As such, performance enhancement needs to be evaluated in a cost-centric context. Further research is needed, and future studies should focus on developing robust solutions that either show improvements across workloads or at least for specific use cases. Furthermore, solutions need to be evaluated considering other metrics, such as incurred cost and additional SSD writes. Improving for one metric alone does not guarantee an improvement.

4.5 Limitations

While our findings infer some clear results, there are several limitations we encountered:

- **Setup** We tested the traces single-threaded, with a fixed DRAM size of 128 MB, no pre-fetching, and no write target rate. Real caching systems operate at a fixed write target and are bigger in size.
- **Workloads** We conducted our tests on six traces from MSR Cambridge. While these traces do represent real I/O traces, we did not test enough. Our traces are also not diverse enough, with sizes ranging only from 10 to 60 GB. Tests need to be conducted on many more workloads with bigger sizes. Moreover, testing more on diverse sources yields more meaningful results.

- **Training** We trained our traces offline, using the author’s designated parameter choices and provided training pipeline. First, using online training could yield better results, which is also indicated by previous research. Furthermore, we used set parameters. Using a variation of parameters could have yielded different results and potentially further improved them.

Chapter 5

Conclusion

5.1 Research questions

The goal of our research was to investigate ML-based solutions for caching, and specifically, in our context, such solutions implemented on top of Cachelib. Based on our results, we can now answer our research questions.

First, we aimed to find the difference between policies in Cachelib. Our results clearly show that such a difference exists. We observed TinyLFU being superior to LRU across workloads, and saw admit-all performing better than reject-first, which, however, comes at the cost of reducing flash lifetime. These results align with the expectations of previous studies, demonstrating the superiority of TinyLFU across various workloads and the trade-off balancing of using reject-first [6].

Our second task in this research was to observe ML-based solutions outperforming heuristic methods. We only found partial support for that. While MAT does improve hit rate almost completely across workloads, other metrics, such as response time and SSD writes, remain variable. Depending on the workload, we can either see improvements or a worsening of performance. While we observe an improvement at least for one metric, and for the one MAT aims to improve, it remains unclear if improving hit rate alone improves overall performance. This was precisely what Yang et al., the authors of Baleen, pointed out. Baleen is supposed to not just focus on one metric, but to improve performance across the board to show a clear sign of an enhancement. Our results only show a small amount of that effect. Depending on the workload, similar to MAT, we observe Baleen to perform in every way, showcasing great workload sensitivity. However, while MAT showed a clear improvement for its target metric,

we again only found partial support for Baleen. Baleen primarily aims to balance backend load and SSD writes. We observe only small reductions in SSD writes, while peak-DT shows no clear improvement across workloads. Thus, we conclude our question with a careful yes, noting that ML-based solutions can indeed enhance performance, but the trade-off and robustness of such solutions remain to be seen.

Lastly, we aim to see if results can be observed across workloads. As already stated, results for heuristic methods remain robust, showcasing a clear difference, and inferring such results will be reproducible in future research. We can only partially say this for ML-based solutions. Even for MAT, which showcases high consistency, we do not see as clear results as for the difference between heuristic methods. Moreover, MAT's results need to be weighed against the incurred cost. For Baleen, we observe the biggest workload sensitivity, with results strongly differing.

5.2 Conclusion

This paper investigates current ML-based solutions for caching, specifically solutions for the caching system Cachelib. As such, we implemented Cachelib and two ML-based solutions on top of it. Our results show that ML can be used to improve performance, but the results are fragile. While solutions can improve metrics, the worsening of other metrics makes results unfit for adaptation. Moreover, other approaches to this problem so far have not shown the desired results. However, results show that improvements are possible, making future research and future adaptation promising.

Bibliography

- [1] B. Berg, D. S. Berger, S. McAllister, I. Grosz, S. Gunasekar, J. Lu, M. Uhlar, J. Carrig, N. Beckmann, M. Harchol-Balter, et al. The {CacheLib} caching engine: Design and experiences at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 753–768, 2020.
- [2] D. S. Berger, S. Henningsen, F. Ciucu, and J. B. Schmitt. Maximizing cache hit ratios by variance reduction. *ACM SIGMETRICS Performance Evaluation Review*, 43(2):57–59, 2015.
- [3] H. Cho and M. Li. Treelite: toolbox for decision tree deployment. 2018.
- [4] M. Chrobak and J. Noga. Lru is better than fifo. *Algorithmica*, 23(2):180–185, 1999.
- [5] D. Duplyakin, R. Ricci, A. Maricq, G. Wong, J. Duerig, E. Eide, L. Stoller, M. Hibler, D. Johnson, K. Webb, A. Akella, K. Wang, G. Ricart, L. Landweber, C. Elliott, M. Zink, E. Cecchet, S. Kar, and P. Mishra. The design and operation of CloudLab. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, pages 1–14, July 2019.
- [6] G. Einziger, R. Friedman, and B. Manes. Tinylfu: A highly efficient cache admission policy. *ACM Transactions on Storage (ToS)*, 13(4):1–31, 2017.
- [7] G. Hasslinger, J. Heikkinen, K. Ntougias, F. Hasslinger, and O. Hohlfeld. Optimum caching versus lru and lfu: Comparison and combined limited look-ahead strategies. In *2018 16th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*, pages 1–6. IEEE, 2018.
- [8] M. Hoseinzadeh, M. Arjomand, and H. Sarbazi-Azad. Reducing access latency of mlc pcms through line striping. *ACM SIGARCH Computer Architecture News*, 42(3):277–288, 2014.
- [9] X. Hu, E. Ramadan, W. Ye, F. Tian, and Z.-L. Zhang. Raven: belady-guided, predictive (deep) learning for in-memory and content caching. In *Proceedings of the 18th Interna-*

- tional Conference on emerging Networking EXperiments and Technologies*, pages 72–90, 2022.
- [10] S. Huang, Q. Wei, D. Feng, J. Chen, and C. Chen. Improving flash-based disk cache with lazy adaptive replacement. *ACM Transactions on Storage (TOS)*, 12(2):1–24, 2016.
- [11] T. Johnson and D. Shasha. X3: A low overhead high performance buffer management replacement algorithm. In *Proceedings of the 20th VLDB Conference*, pages 439–450, 1994.
- [12] S. McAllister, B. Berg, J. Tutuncu-Macias, J. Yang, S. Gunasekar, J. Lu, D. S. Berger, N. Beckmann, and G. R. Ganger. Kangaroo: Caching billions of tiny objects on flash. In *Proceedings of the ACM SIGOPS 28th symposium on operating systems principles*, pages 243–262, 2021.
- [13] D. Narayanan, A. Donnelly, and A. Rowstron. Write off-loading: Practical power management for enterprise storage. *ACM Transactions on Storage (TOS)*, 4(3):1–23, 2008.
- [14] S. Pan, T. Stavrinou, Y. Zhang, A. Sikaria, P. Zakharov, A. Sharma, M. Shuey, R. Wareing, M. Gangapuram, G. Cao, et al. Facebook’s tectonic filesystem: Efficiency from exascale. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 217–231, 2021.
- [15] S. Podlipnig and L. Böszörményi. A survey of web cache replacement strategies. *ACM Computing Surveys (CSUR)*, 35(4):374–398, 2003.
- [16] A. Press, M. Pacer, T. Griffiths, and B. Christian. Caching algorithms and rational models of memory. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, volume 36, 2014.
- [17] L. V. Rodriguez, F. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, and G. Narasimhan. Learning cache replacement with {CACHEUS}. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 341–354, 2021.
- [18] E. A. Shriver, C. Small, and K. A. Smith. Why does file system prefetching work? In *USENIX Annual Technical Conference, General Track*, pages 71–84, 1999.
- [19] A. J. Smith. Cache memories. *ACM Computing Surveys (CSUR)*, 14(3):473–530, 1982.
- [20] Z. Song, D. S. Berger, K. Li, and W. Lloyd. Learning relaxed belady for content distribution network caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 529–544, 2020.

- [21] Z. Song, K. Chen, N. Sarda, D. Altınbüken, E. Brevdo, J. Coleman, X. Ju, P. Jurczyk, R. Schooler, and R. Gummadi. {HALP}: Heuristic aided learned preference eviction policy for {YouTube} content delivery network. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1149–1163, 2023.
- [22] G. Vietri, L. V. Rodriguez, W. A. Martinez, S. Lyons, J. Liu, R. Rangaswami, M. Zhao, and G. Narasimhan. Driving cache replacement with {ML-based}{LeCaR}. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*, 2018.
- [23] D. L.-K. Wong, H. Wu, C. Molder, S. Gunasekar, J. Lu, S. Khandkar, A. Sharma, D. S. Berger, N. Beckmann, and G. R. Ganger. Baleen:{ML} admission & prefetching for flash caches. In *22nd USENIX Conference on File and Storage Technologies (FAST 24)*, pages 347–371, 2024.
- [24] D. Yang, D. S. Berger, K. Li, and W. Lloyd. A learned cache eviction framework with minimal overhead. *arXiv preprint arXiv:2301.11886*, 2023.
- [25] J. Yang, Z. Mao, Y. Yue, and K. Rashmi. {GL-Cache}: Group-level learning for efficient and high-performance caching. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*, pages 115–134, 2023.
- [26] T.-W. Yang, S. Pollen, M. Uysal, A. Merchant, and H. Wolfmeister. {CacheSack}: Admission optimization for google datacenter flash caches. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 1021–1036, 2022.